

VOLUMEN IV · LIBRO 4 DE 5

BMAD Catálogo de Skills

Referencia comando a comando: el diccionario del método

Ficha completa de las 32 skills del flujo del proyecto:
propósito, prerequisites, salidas, sub-pasos y duración.
Apéndice con las 34 skills complementarias.

BMAD Method · CIS · TEA · BMB · Core

2026-05-07 · LIDR.co

0 Bienvenida

Este libro es la referencia comando a comando del método. No está pensado para leerse de principio a fin: se consulta cuando vas a invocar una skill concreta y necesitas saber qué espera, qué produce y cuánto tarda.

El método incluye **66 skills** en total. Este catálogo cubre las **32 que componen el flujo del proyecto** (módulos BMM y TEA) con detalle pleno, y deja las otras 34 como apéndice resumido (módulos CIS, BMB, Core, agentes con persona y skills auxiliares de BMM).

0.1 Cómo está organizado el catálogo

Sección	Contenido	Cuándo la usas
Sec. 1 Convenciones	Anatomía de la ficha de skill, módulos, insignias.	Hojear la primera vez para entender el formato.
Sec. 2 Catálogo detallado	32 fichas, una por skill, agrupadas por fase del flujo.	Antes de invocar una skill concreta.
Sec. 3 Apéndice	34 skills complementarias en una línea cada una.	Cuando una skill no aparece en Sec. 2 y quieres saber para qué sirve.
Sec. 4 Verificación	Versión BMAD verificada, fuentes consultadas.	Para auditar que el catálogo está al día.

0.2 ¿Qué es una skill?

Una **skill** es un workflow ejecutable que sigue una serie de pasos estructurados para producir un artefacto concreto (un documento, un test, una decisión, una refactorización). Se invoca con un `/comando` en el agente de IA.

En el sistema instalado, cada skill vive en `.claude/skills/<nombre-de-la-skill>/`. Dentro hay un archivo principal (`SKILL.md`) que orquesta la ejecución y, en muchas skills, una carpeta de sub-pasos. La carpeta `_bmad/` contiene la configuración del método (manifests, módulos), no las skills en sí.

MODOS CEV EN LAS SKILLS TEA
Las nueve skills TEA usan un patrón uniforme con tres modos: **Create** (genera el artefacto desde cero, en `steps-c/`), **Edit** (modifica un artefacto existente, en `steps-e/`) y **Validate** (audita el artefacto sin tocarlo, en `steps-v/`). El modo se selecciona automáticamente según el contexto, o se fuerza con un argumento.

0.3 Anatomía de una ficha

Cada ficha del Sec. 2 sigue siempre el mismo orden, para que el lector localice la información en el mismo sitio sin tener que leerla entera:

- Cabecera** — comando, módulo, fase y obligatoriedad.
- Resuelve** — el problema que aborda en una sola frase.
- Cuándo invocarla** — tres situaciones concretas.
- Necesita antes** — prerequisites verificables.

- 5. **Produce** — artefacto de salida y dónde se guarda.
- 6. **Cómo funciona** — sub-pasos o fases internas.
- 7. **Cuánto tarda** — rango orientativo.
- 8. **Consultar también** — skills relacionadas (cuando aplica).

1 Convenciones

1.1 Punto de color del módulo

- **BMM** · flujo principal
- **TEA** · testing
- **CIS** · pensamiento creativo
- **BMB** · extensión del método
- **Core** · utilidades transversales

1.2 Insignias de obligatoriedad

- REQ** obligatoria para avanzar de fase · **REC** recomendada fuertemente · **OPC** opcional según necesidad · **REQ TEA** obligatoria si el equipo adopta TEA.

1.3 Insignias de aporte LIDR

- VISIÓN LIDR** framing conceptual nuestro · **RECOMENDACIÓN LIDR** sugerencia operativa basada en nuestra experiencia.

2 Catálogo detallado — 32 skills del flujo del proyecto

Las skills se presentan en orden natural de invocación: Fase 0 (aprendizaje opcional) → Fase 1 → Fase 2 → Fase 3 → Fase 4 → toolkit anytime → brownfield.

2.0 Fase 0 — Aprendizaje (opcional, solo si se adopta TEA)

01

/bmad-teach-me-testing

● Test Architect

OPC

FASE 0

RESUELVE: Forma al equipo en testing antes de adoptar TEA mediante un programa autodirigido con persistencia de progreso entre sesiones.

CUÁNDO INVOCARLA

- El equipo adopta TEA por primera vez.
- Onboarding de Dev o QA al stack de testing.
- Antes de `/bmad-testarch-test-design` si los conceptos base no están claros.

PRODUCE

Notas de cada sesión + archivo de progreso reanudable.
Al final, certificado de finalización del programa.

CUÁNTO TARDA

2-4 h por sesión × 7 sesiones = entre 14 y 28 h,
distribuidas en una o dos semanas.

NECESITA ANTES

- Configuración del módulo TEA (idioma, nombre del proyecto).
- No depende de PRD ni arquitectura.

CÓMO FUNCIONA

12 sub-pasos en modo *Create*, de los que 7 son las sesiones académicas (`step-04-session-01` a `session-07`). Incluye modos *Validate* y *Edit* para auditar o modificar el progreso.

CONSULTAR TAMBIÉN

Libro 5 (Protocolo TEA) para la estrategia de testing en proyecto real.

2.1 Fase 1 — Análisis

02

`/bmad-brainstorming`

● Core / BMM

OPC

FASE 1

RESUELVE: Genera más de cien ideas estructuradas mediante un catálogo de técnicas creativas con protocolo anti-sesgo que pivota dominio cada diez ideas.

CUÁNDO INVOCARLA

- Greenfield sin claridad de alcance.
- Bloqueo del equipo generando opciones.
- Antes de `/bmad-product-brief` si la idea aún es difusa.

PRODUCE

Archivo append-only por sesión, con marca de tiempo, en `{output}/brainstorming/`. Conserva todo el historial; se puede retomar.

CUÁNTO TARDA

60-120 minutos.

NECESITA ANTES

- Configuración del módulo (idioma, carpeta de salida, nivel del usuario).
- Opcional: archivo de contexto previo.

CÓMO FUNCIONA

Ocho sub-pasos (setup, continue, técnica seleccionada por usuario, técnica recomendada, técnica aleatoria, flujo progresivo, ejecución, organización). Catálogo de **60 técnicas** distintas.

CONSULTAR TAMBIÉN

`/bmad-cis-design-thinking` y `/bmad-cis-innovation-strategy` para enfoques alternativos.

03

`/bmad-product-brief`

● BMad Method

REC

FASE 1

RESUELVE: Produce un brief de producto persuasivo de una a dos páginas, listo para alimentar el PRD, mediante descubrimiento colaborativo y análisis de artefactos previos.

CUÁNDO INVOCARLA

- Tras brainstorm y research, antes del PRD.
- Para alinear stakeholders en una página.
- Para refrescar un brief antiguo sin reescribirlo desde cero.

PRODUCE

Brief ejecutivo en la carpeta de planificación.

Opcionalmente, un resumen comprimido optimizado para alimentar al LLM al crear el PRD.

CUÁNTO TARDA

Guiado: 45–90 min. Rápido y autónomo: ~30 min.

NECESITA ANTES

- Idea inicial o brief existente.
- Opcional: outputs previos de brainstorming, market o domain research.

CÓMO FUNCIONA

Cinco etapas (entender intención, descubrimiento contextual, elicitación guiada, redacción y revisión, finalización). Tres modos: *guiado*, *rápido* y *autónomo*.

CONSULTAR TAMBIÉN

`/bmad-prfaq` (apéndice) como alternativa cuando la idea aún no está validada.

04

`/bmad-market-research`

● BMad Method

OPC

FASE 1

RESUELVE: Investigación de mercado con búsqueda web obligatoria y citas verificables; cubre comportamiento del cliente, problemas, factores de decisión y análisis competitivo.

CUÁNDO INVOCARLA

- Necesidad de datos actualizados sobre competencia o mercado.
- Antes del Product Brief en sectores cambiantes.
- Cuando el cliente pide validación de mercado pre-PRD.

PRODUCE

Informe en

`{planning}/research/market-{tema}-{fecha}.md`

con narrativa estructurada y citas verificables.

CUÁNTO TARDA

90–120 minutos.

NECESITA ANTES

- Tema de investigación definido.
- Acceso a búsqueda web (la skill aborta si no está disponible).

CÓMO FUNCIONA

Seis sub-pasos (init, comportamiento del cliente, problemas, decisiones, análisis competitivo, finalización).

CONSULTAR TAMBIÉN

`/bmad-domain-research` para foco regulatorio y de sector.

05

`/bmad-domain-research`

● BMad Method

OPC

FASE 1

RESUELVE: Mapea la industria entera (regulación, tendencias técnicas, panorama competitivo) en lugar del producto específico. Crítica en sectores regulados.

CUÁNDO INVOCARLA

- Industria nueva para el equipo.
- Mapear contexto regulatorio antes de PRD.
- Generar informe de sector pre-presupuesto.

PRODUCE

Informe en

`{planning}/research/domain-{tema}-{fecha}.md`

CUÁNTO TARDA

90–120 minutos.

NECESITA ANTES

- Dominio o industria definido.
- Acceso a búsqueda web.

CÓMO FUNCIONA

Seis sub-pasos (init, análisis de dominio, panorama competitivo, foco regulatorio, tendencias técnicas, síntesis).

CONSULTAR TAMBIÉN

`/bmad-market-research` para foco en cliente y competencia directa.

06

`/bmad-technical-research`

● BMad Method

OPC

FASE 1

RESUELVE: Spike técnico breve basado en evidencia web: evaluación de tecnologías, comparativa de arquitecturas y trade-offs antes de tomar una decisión cargada.

CUÁNDO INVOCARLA

- Decisión arquitectónica con dos o tres candidatos serios.
- Antes de `/bmad-create-architecture`.
- Migraciones o upgrades mayores.

PRODUCE

Informe en

`{planning}/research/technical-{tema}-{fecha}.md`

con comparativa estructurada y trade-offs.

CUÁNTO TARDA

90–120 minutos.

NECESITA ANTES

- Tecnologías candidatas listadas.
- Acceso a búsqueda web.

CÓMO FUNCIONA

Seis sub-pasos paralelos al market research, adaptados a contexto técnico.

CONSULTAR TAMBIÉN

`/bmad-create-architecture` consume directamente el informe.

2.2 Fase 2 — Planificación

07

`/bmad-create-prd`

● BMad Method

REQ

FASE 2

RESUELVE: Produce el PRD¹ exhaustivo del producto desde cero, con seguimiento de progreso entre sesiones.

1. PRD: *Product Requirements Document*. Documento que describe lo que el producto debe hacer (requisitos funcionales) y cómo de bien debe hacerlo (requisitos no funcionales). Es el contrato entre producto y equipo técnico.

CUÁNDO INVOCARLA

- Greenfield: primer PRD del proyecto.
- Brownfield: épica grande con alcance independiente.
- Refresh completo de un PRD obsoleto.

PRODUCE

`{planning}/prd.md`. Incluye seguimiento de pasos completados en el frontmatter para reanudar.

CUÁNTO TARDA

3-5 horas distribuidas en dos o tres sesiones.

NECESITA ANTES

- Product Brief.
- Recomendado: market y domain research.

CÓMO FUNCIONA

15 sub-pasos en `steps-c/`: init, continue, discovery, vision, executive summary, success criteria, journeys, domain, innovation, project type, scoping, FRs, NFRs, polish, complete.

CONSULTAR TAMBIÉN

`/bmad-validate-prd` al terminar; `/bmad-edit-prd` para correcciones posteriores.

08

`/bmad-validate-prd`

● BMad Method

REC

FASE 2

RESUELVE: Audita el PRD contra trece dimensiones de calidad y produce un reporte de incidencias priorizadas.

CUÁNDO INVOCARLA

- Tras `/bmad-create-prd`, antes de arquitectura o UX.
- Revisión de un PRD escrito por otra persona.
- Antes de `/bmad-check-implementation-readiness`.

PRODUCE

Reporte en formato markdown con incidencias categorizadas por dimensión y recomendaciones priorizadas.

CUÁNTO TARDA

60-90 minutos.

NECESITA ANTES

- Path al PRD a validar.

CÓMO FUNCIONA

14 sub-pasos de validación en `steps-v/`: descubrimiento, formato, paridad, densidad, cobertura del brief, medibilidad, trazabilidad, intrusión de implementación, dominio, tipo de proyecto, heurísticas, calidad holística, completitud, reporte.

CONSULTAR TAMBIÉN

`/bmad-edit-prd` para resolver las incidencias detectadas.

RECOMENDACIÓN LIDR

El método marca `/bmad-validate-prd` como opcional. Recomendamos tratarla como recomendada porque saltarla es la causa más frecuente de retrabajo en Fase 4: los huecos del PRD viajan al backlog y se hacen visibles cuando el equipo ya está implementando. Para ganar objetividad, conviene ejecutarla con un LLM distinto al que produjo el PRD.

09

`/bmad-create-ux-design`

● BMad Method

REC SI UI

FASE 2

RESUELVE: Especificación UX completa: descubrimiento, experiencia central, sistema de diseño, viajes de usuario, estrategia de componentes, patrones, accesibilidad y diseño responsivo.

CUÁNDO INVOCARLA

- Producto con interfaz de usuario relevante.
- Tras PRD aprobado, antes de la arquitectura.
- Saltar si el producto es 100 % backend, CLI o API.

PRODUCE

`{planning}/ux-design-specification.md` basado en plantilla.

CUÁNTO TARDA

4-6 horas distribuidas.

NECESITA ANTES

- PRD existente o, en su defecto, Product Brief.

CÓMO FUNCIONA

15 sub-pasos: init, continue, discovery, experiencia central, respuesta emocional, inspiración, sistema de diseño, experiencia que define al producto, fundamentos visuales, direcciones de diseño, viajes de usuario, estrategia de componentes, patrones de UX, responsivo y accesibilidad, completar.

CONSULTAR TAMBIÉN

Libro 3 (Plan de Sesiones) para encajarla en la sesión correspondiente.

10

`/bmad-testarch-nfr`

● Test Architect

REC

FASE 2 & 4

RESUELVE: Evalúa los requisitos no funcionales (rendimiento, seguridad, fiabilidad, escalabilidad) con validación basada en evidencia y desenlace determinista.

CUÁNDO INVOCARLA

- En Fase 2 (modo *setup*) para fijar umbrales en el PRD.
- Mid-ciclo (modo *check*) para detectar desviaciones.
- Pre-release (modo *gate*) como decisión final.

PRODUCE

Reporte por categoría con scoring, evidencia recogida y recomendaciones. Modo *gate* emite firma final: PASS, CONCERNS, WAIVED o FAIL.

CUÁNTO TARDA

50-75 minutos por invocación.

NECESITA ANTES

- PRD con NFR² identificados.
- Para *check* y *gate*: evidencia (load tests, security scans, métricas de fiabilidad).

CÓMO FUNCIONA

Modo *Create*: 11 sub-pasos con sub-agentes paralelos por categoría (security, performance, reliability, scalability). Modos *Validate* y *Edit*: dos pasos cada uno.

CONSULTAR TAMBIÉN

Libro 5 (Protocolo TEA) para el detalle de cada categoría.

VISIÓN LIDR

En el método, NFR es una sola skill con tres modos. Nosotros la pensamos como tres momentos del proyecto — setup en Fase 2, check a mitad del ciclo, gate antes del release — porque cada momento responde una pregunta distinta: setup fija el contrato, check vigila la deriva, gate firma la salida. El catálogo solo lista la skill una vez; las invocaciones son tres.

2.3 Fase 3 — Solución

11

`/bmad-create-architecture`

● BMad Method

REQ

FASE 3

2. NFR: *Non-Functional Requirement*. Requisito que describe cómo de bien debe comportarse el sistema (rendimiento, seguridad, fiabilidad, escalabilidad), en contraposición a los *functional requirements* que describen qué debe hacer.

RESUELVE: Documenta las decisiones arquitectónicas para asegurar consistencia entre desarrolladores y agentes IA: patrones, estructura de carpetas y convenciones.

CUÁNDO INVOCARLA

- Tras PRD aprobado, antes de épicas e historias.
- Cambios arquitectónicos mayores (refactor o migración de stack).
- Onboarding de agentes IA al proyecto.

PRODUCE

`{planning}/architecture.md` con decisiones técnicas, patrones y restricciones.

CUÁNTO TARDA

60–90 minutos.

NECESITA ANTES

- PRD completado.
- Recomendado: technical research previo si la decisión es cargada.

CÓMO FUNCIONA

9 sub-pasos: init, continue, contexto, starter, decisiones, patrones, estructura, validación, complete.

CONSULTAR TAMBIÉN

`/bmad-testarch-test-design` después, para validar testabilidad.

12

`/bmad-testarch-test-design`

● Test Architect

REQ TEA

FASE 3

RESUELVE: Plan de testing basado en riesgo con clasificación de prioridades (P0–P3), criterios de entrada y salida y orden de ejecución.

CUÁNDO INVOCARLA

- Pre-gate de la fase de solución, para validar testabilidad arquitectónica.
- Por cada épica grande en Fase 4 (modo epic-level).
- Para fijar criterios de entrada y salida con QA.

PRODUCE

Plan de testing y matriz de riesgos. Plantillas: system, epic, qa, handoff y architecture.

CUÁNTO TARDA

30–45 minutos.

NECESITA ANTES

- System-level: arquitectura y ADR³.
- Epic-level: épica + historias + PRD.

CÓMO FUNCIONA

Modo *Create* con 6 sub-pasos: detect mode, load context, riesgos y testabilidad, plan de cobertura, generación de salida, resume. Auto-detecta si está en system-level o epic-level.

CONSULTAR TAMBIÉN

`/bmad-testarch-framework` después, en cadena estricta.

13

`/bmad-testarch-framework`

● Test Architect

REQ TEA

FASE 3

RESUELVE: Andamiaje del framework de tests listo para producción (Playwright o Cypress) con fixtures, helpers, configuración y buenas prácticas.

3. ADR: *Architecture Decision Record*. Documento corto que captura una decisión técnica (qué se eligió, qué se descartó y por qué), con sus trade-offs. Es el rastro auditable de las decisiones de arquitectura, distinto del documento general de arquitectura.

CUÁNDO INVOCARLA

- Inicializar testing en proyecto nuevo o repositorio sin tests.
- Migración entre frameworks.
- Tests sueltos sin estructura coherente.

PRODUCE

`playwright.config.ts` o `cypress.config.ts`; fixtures, helpers, page objects, scripts; README de setup.

CUÁNTO TARDA

25–35 minutos.

NECESITA ANTES

- Test Design ejecutado.
- Stack tecnológico decidido.
- Acceso de escritura al repositorio.

CÓMO FUNCIONA

Modo *Create* con 6 sub-pasos: preflight, selección de framework, scaffolding, docs y scripts, validación, resume. Configurable por preferencia, tamaño del proyecto y uso de TypeScript.

CONSULTAR TAMBIÉN

`/bmad-testarch-ci` después, para enlazar la pipeline.

14

`/bmad-testarch-ci`

● Test Architect

REQ TEA

FASE 3

RESUELVE: Pipeline de integración continua multiplataforma con ejecución de tests, ciclos de *burn-in* para detectar tests inestables, sharding paralelo, recolección de artefactos y notificaciones.

CUÁNDO INVOCARLA

- Tras framework configurado, antes de Fase 4.
- Pipeline existente sin gates de calidad.
- Para detectar y aislar tests inestables.

PRODUCE

Pipeline específico de la plataforma: GitHub Actions, GitLab CI, Azure Pipelines, Jenkins o Harness. Incluye gates de calidad, recolección de artefactos y plantillas de notificación.

CUÁNTO TARDA

20–30 minutos.

NECESITA ANTES

- Framework instalado.
- Plataforma de CI elegida.
- Acceso al repositorio.

CÓMO FUNCIONA

Modo *Create* con 5 sub-pasos: preflight, generación de pipeline, configuración de gates, validación, resume.

CONSULTAR TAMBIÉN

`/bmad-testarch-atdd` en Fase 4 ya con la pipeline activa.

15

`/bmad-create-epics-and-stories`

● BMad Method

REQ

FASE 3

RESUELVE: Descompone el PRD y la arquitectura en épicas e historias organizadas por valor, con criterios de aceptación Given/When/Then⁴, requisitos técnicos y pistas de implementación.

4. Given/When/Then: estructura de criterios de aceptación que describe un escenario en tres partes — dado un contexto inicial (Given), cuando ocurre un evento (When), entonces se espera un resultado (Then).

CUÁNDO INVOCARLA

- Tras PRD y arquitectura aprobados.
- Refresh del backlog cuando el alcance del PRD cambia.

PRODUCE

`{planning}/epics.md` o, si el equipo lo prefiere, varias `epics/*.md` divididas. Cada historia con criterios de aceptación, requisitos técnicos, dependencias y pistas de origen.

CUÁNTO TARDA

45–60 minutos.

NECESITA ANTES

- PRD completado.
- Arquitectura completada.

CÓMO FUNCIONA

4 sub-pasos: validación de prerequisites, diseño de épicas, creación de historias, validación final.

CONSULTAR TAMBIÉN

`/bmad-check-implementation-readiness` como gate inmediato después.

16

`/bmad-check-implementation-readiness`

● BMad Method

REQ

FASE 3

RESUELVE: Audita que los cuatro documentos de planificación (PRD, UX, arquitectura, épicas) están completos, alineados entre sí y listos para implementar.

CUÁNDO INVOCARLA

- Justo antes de Fase 4 — gate obligatorio.
- Tras cambios significativos en cualquiera de los cuatro documentos.

PRODUCE

Reporte de readiness con huecos detectados, severidad y recomendaciones de resolución.

CUÁNTO TARDA

60–90 minutos.

NECESITA ANTES

- PRD, arquitectura, épicas e historias y, si aplica, UX, todos completados.

CÓMO FUNCIONA

6 sub-pasos: descubrimiento de documentos, análisis del PRD, validación de cobertura por épica, alineación con UX, revisión de calidad de épicas, evaluación final.

CONSULTAR TAMBIÉN

`/bmad-correct-course` si el reporte detecta huecos significativos.

2.4 Fase 4 — Implementación

17

`/bmad-sprint-planning`

● BMad Method

REQ

FASE 4 · INICIO

RESUELVE: Genera el archivo `sprint-status.yaml` a partir del documento de épicas. Construye la máquina de estados que el resto de la fase consume.

CUÁNDO INVOCARLA

- Inicio de Fase 4.
- Re-sincronización tras cambios en `epics.md`.

PRODUCE

`{implementation}/sprint-status.yaml` con el estado actual detectado para cada historia (backlog, lista para dev, en progreso, en revisión, hecha).

CUÁNTO TARDA

15–20 minutos.

NECESITA ANTES

- `epics.md` o `epics/*.md` completados.

CÓMO FUNCIONA

Cinco fases: parseo de épicas, construcción de la estructura, detección inteligente del estado actual, generación del YAML, validación y reporte. Sin carpeta de sub-pasos: la lógica vive en `SKILL.md` y la plantilla.

CONSULTAR TAMBIÉN

`/bmad-sprint-status` para consultar el estado a lo largo del sprint.

18

`/bmad-create-story`

● BMad Method

REQ

FASE 4 · CICLO

RESUELVE: Crea el archivo de la historia con todo el contexto que el agente Dev necesita: análisis del PRD, arquitectura, UX, historias previas, historial de Git e investigación web actualizada.

CUÁNDO INVOCARLA

- Antes de cada `/bmad-dev-story`.
- Para preparar la siguiente historia del backlog.
- Para crear una historia específica solicitada por el usuario.

PRODUCE

`{implementation}/{story-key}.md` con requisitos, criterios de aceptación, tareas, notas técnicas, inteligencia de Git, investigación web y referencia a `project-context.md`.

CUÁNTO TARDA

40–60 minutos.

NECESITA ANTES

- `epics.md`, `prd.md`, `architecture.md`, UX si aplica.
- `sprint-status.yaml`.
- Acceso al historial de Git.

CÓMO FUNCIONA

Dos acciones: **create** (REQ) genera la siguiente historia y **validate** (REC) verifica su completitud antes del desarrollo. Seis fases: determinar objetivo, cargar artefactos, análisis arquitectónico, investigación web, creación del archivo, actualización del sprint-status. Sin carpeta de sub-pasos.

CONSULTAR TAMBIÉN

`/bmad-testarch-atdd` después, en cadena estricta.

19

`/bmad-testarch-atdd`

● Test Architect

REQ TEA

FASE 4 · CICLO

RESUELVE: Genera tests de aceptación que fallan (fase roja del ciclo red-green-refactor⁵) antes de la implementación, en niveles apropiados (E2E, API, componente).

5. red-green-refactor: ciclo de Test-Driven Development en tres pasos. Rojo: escribir un test que falle. Verde: implementar lo mínimo para que el test pase. Refactor: limpiar el código manteniendo el test verde.

CUÁNDO INVOCARLA

- Tras `/bmad-create-story`, antes de `/bmad-dev-story`.
- Cuando el equipo quiere ATDD/TDD estricto.
- Para convertir criterios de aceptación en tests ejecutables.

PRODUCE

Tests fallidos por nivel + fixtures + helpers; checklist de implementación; trazabilidad criterios↔tests.

CUÁNTO TARDA

40–60 minutos.

NECESITA ANTES

- Historia con criterios de aceptación y requisitos técnicos.
- Framework configurado (`/bmad-testarch-framework`).

CÓMO FUNCIONA

Modo *Create* con 9 sub-pasos y sub-agentes paralelos por nivel (API, E2E). Modos *Validate* y *Edit* disponibles.

CONSULTAR TAMBIÉN

`/bmad-dev-story` implementa lo que estos tests piden (fase verde).

20

`/bmad-dev-story`

● BMad Method

REQ

FASE 4 · CICLO

RESUELVE: Implementa la historia siguiendo su archivo de especificación con el ciclo red-green-refactor: tareas, tests, validaciones y marca la historia como `review`.

CUÁNDO INVOCARLA

- Historia en estado *ready-for-dev*.
- Continuación tras code review (cuando vuelve con cambios).
- Tras tests ATDD en rojo.

PRODUCE

Código nuevo o modificado, tests unitarios y de integración, archivo de historia actualizado (tareas marcadas, change log, lista de archivos, estado *review*) y `sprint-status` actualizado.

CUÁNTO TARDA

120–180 minutos por historia (variable según complejidad).

NECESITA ANTES

- Archivo de historia con tareas, criterios y notas.
- `sprint-status.yaml`.
- Repositorio Git en estado limpio o committable.

CÓMO FUNCIONA

Diez fases: localizar la siguiente historia lista, cargar contexto, detectar continuación tras review, marcar en progreso, implementar red-green-refactor, autorar tests, ejecutar validaciones, marcar completa, finalizar, comunicar. Sin carpeta de sub-pasos.

CONSULTAR TAMBIÉN

`/bmad-testarch-automate` después;
`/bmad-code-review` antes del merge.

21

`/bmad-testarch-automate`

● Test Architect

REC

FASE 4 · CICLO

RESUELVE: Expansión de la cobertura tras la implementación: genera tests priorizados a nivel E2E, API, componente y unitario usando sub-agentes paralelos.

CUÁNDO INVOCARLA

- Tras `/bmad-dev-story` y `/bmad-code-review`, para ampliar más allá de los criterios de aceptación.
- En proyectos brownfield que necesitan cobertura retroactiva.
- Para alcanzar un umbral de cobertura antes del release.

PRODUCE

Tests por nivel + reporte de cobertura (actual contra objetivo); fixtures, helpers y cola de implementación.

CUÁNTO TARDA

45-60 minutos.

NECESITA ANTES

- Codebase existente.
- Modo *BMad-Integrated*: historia, PRD y test design.
Modo *Standalone*: solo código.

CÓMO FUNCIONA

Modo *Create* con 9 sub-pasos: preflight, identificar objetivos, generar tests, sub-agentes API, backend y E2E, agregar, validar, resume.

CONSULTAR TAMBIÉN

`/bmad-testarch-test-review` para auditar la calidad de los tests acumulados.

22

`/bmad-code-review`

● BMad Method

REC

FASE 4 · CICLO

RESUELVE: Revisión adversarial pre-merge usando tres capas paralelas: *Blind Hunter* (busca bugs sin contexto), *Edge Case Hunter* (recorre ramas y condiciones límite) y *Acceptance Auditor* (valida cumplimiento de criterios de aceptación).

CUÁNDO INVOCARLA

- Tras `/bmad-dev-story` con la historia en review.
- Antes de mergear a la rama principal.
- Cuando hay dudas sobre la calidad del cambio.

PRODUCE

Sección "Senior Developer Review (AI)" añadida al archivo de historia, con triage en categorías de severidad (alta, media, baja) y lista de seguimiento accionable.

CUÁNTO TARDA

30-50 minutos.

NECESITA ANTES

- Archivo de historia con lista de archivos modificados y criterios de aceptación.
- Código implementado.

CÓMO FUNCIONA

4 sub-pasos: recolectar contexto, revisar (las tres capas paralelas), triar, presentar.

CONSULTAR TAMBIÉN

Recomendamos usar un LLM distinto al que implementó para máxima objetividad.

RECOMENDACIÓN LIDR

El método marca `/bmad-code-review` como opcional. Recomendamos tratarlo como recomendado del ciclo: el code review captura defectos antes del merge usando un LLM distinto al que implementó. Saltarlo abarata el ciclo a corto plazo y lo encarece a medio plazo, porque los defectos llegan a la pipeline o a producción.

2.4.1 CIERRE DE ÉPICA

23

`/bmad-testarch-test-review`

● Test Architect

OPC

FASE 4 · CIERRE

RESUELVE: Auditoría de calidad del conjunto de tests con puntuación de 0 a 100 y hallazgos accionables por dimensión: determinismo, aislamiento, mantenibilidad y rendimiento.

CUÁNDO INVOCARLA

- Sospechas de tests inestables o lentos.
- Cierre de épica (paso 1 del cierre formal).
- Pre-release o auditoría trimestral.

PRODUCE

Reporte de calidad con puntuación 0–100 y hallazgos por dimensión.

CUÁNTO TARDA

40–55 minutos.

NECESITA ANTES

- Suite de tests escrita.

CÓMO FUNCIONA

Modo *Create* con 10 sub-pasos y sub-agentes especializados por dimensión (determinismo, aislamiento, mantenibilidad, rendimiento).

CONSULTAR TAMBIÉN

`/bmad-testarch-trace` para cobertura de requisitos (queda fuera del scope de esta skill).

24

`/bmad-testarch-trace`

● Test Architect

OPC

FASE 4 · CIERRE

RESUELVE: Matriz de trazabilidad requisitos↔tests + análisis de huecos + decisión opcional de gate de calidad (PASS / CONCERNS / FAIL / WAIVED) basada en evidencia. Salida go/no-go antes del release.

CUÁNDO INVOCARLA

- Cierre de épica (paso 2, después del test review).
- Pre-release.
- Auditoría regulatoria.

PRODUCE

Matriz de trazabilidad + reporte de cobertura + decisión de gate.

CUÁNTO TARDA

25–35 minutos.

NECESITA ANTES

- Requisitos documentados (PRD + épicas).
- Suite de tests implementado.
- Recomendado: output de `/bmad-testarch-test-review`.

CÓMO FUNCIONA

Modo *Create* con 6 sub-pasos: load context, descubrir tests, mapear criterios, analizar huecos, decisión de gate, resume.

CONSULTAR TAMBIÉN

`/bmad-retrospective` después, para cerrar formalmente la épica.

25

`/bmad-retrospective`

● BMad Method

OPC

FASE 4 · CIERRE

RESUELVE: Retrospectiva formal tras épica: extrae lecciones aprendidas, evalúa el éxito y prepara la próxima épica. Formato conversacional multi-agente facilitado por el Scrum Master.

CUÁNDO INVOCARLA

- Tras completar todas las historias de una épica + test review + traceability.
- Final de un sprint largo con aprendizajes acumulados.

PRODUCE

`{implementation}/epic-{N}-retrospective.md`

con lecciones aprendidas, métricas de éxito, action items con responsable y preparación de la siguiente épica.

CUÁNTO TARDA

45–75 minutos.

NECESITA ANTES

- `sprint-status.yaml` y archivos de historias.
- Recomendado: outputs de test review y trace como input concreto.

CÓMO FUNCIONA

Cuatro fases: descubrimiento de la épica, facilitación en formato party mode, revisión, preparación de la próxima épica. Sin carpeta de sub-pasos.

CONSULTAR TAMBIÉN

`/bmad-correct-course` si aparecen issues mayores en la retro.

2.5 Toolkit anytime — correcciones y consulta

Skills invocables en cualquier momento del ciclo, una vez producidos los artefactos correspondientes. Resuelven necesidades puntuales sin obligar a planificar nueva sesión.

26

`/bmad-edit-prd`

● BMad Method

OPC

ANYTIME · POST-PRD

RESUELVE: Edición guiada de un PRD existente para resolver incidencias detectadas por la validación o aplicar cambios de alcance.

CUÁNDO INVOCARLA

- Resolver issues de `/bmad-validate-prd` sistemáticamente.
- Cambio de alcance que afecta varias secciones.
- Migrar un PRD antiguo al formato actual.

PRODUCE

PRD actualizado en su path original, con seguimiento de pasos completados.

CUÁNTO TARDA

45–90 minutos.

NECESITA ANTES

- Path al PRD existente.
- Opcional: reporte de `/bmad-validate-prd`.

CÓMO FUNCIONA

Modo *Edit* con **5 sub-pasos**: descubrimiento, conversión legacy, revisión, edición, completar.

CONSULTAR TAMBIÉN

`/bmad-validate-prd` antes y después para verificar el efecto.

27

`/bmad-correct-course`

● BMad Method

OPC

ANYTIME · MID-SPRINT

RESUELVE: Gestión de cambios significativos durante la implementación. Analiza el impacto en PRD, épicas, arquitectura y UX, y produce una propuesta estructurada de cambio con handoff.

CUÁNDO INVOCARLA

- Requisito o restricción nueva mid-sprint.
- Bug crítico que fuerza re-priorización.
- Ajuste solicitado por stakeholder.

PRODUCE

`{planning}/sprint-change-proposal-{fecha}.md`

con análisis de impacto, cambios propuestos y handoff. Posibles updates a épicas, historias y PRD.

CUÁNTO TARDA

60–90 minutos.

NECESITA ANTES

- PRD y épicas presentes (la skill aborta si faltan).
- Descripción del trigger del cambio.

CÓMO FUNCIONA

Seis fases: inicializar navegación, ejecutar checklist de análisis, redactar propuestas, generar Sprint Change Proposal, finalizar y enrutar, completar.

CONSULTAR TAMBIÉN

`/bmad-edit-prd` y `/bmad-create-story` según las decisiones que salgan.

28

`/bmad-sprint-status`

● BMad Method

OPC

ANYTIME · CONSULTA

RESUELVE: Resumen rápido del estado del sprint con visibilidad accionable y recomendación de la siguiente acción.

CUÁNDO INVOCARLA

- Antes de empezar el día o antes de `/bmad-create-story`.
- Cierre de sprint o de épica.
- Tras pausas largas para reorientar al equipo.

PRODUCE

Reporte conversacional con progreso, próxima acción y riesgos detectados. Modo *data*: salida JSON para scripts o pipelines.

CUÁNTO TARDA

5–10 minutos.

NECESITA ANTES

- `sprint-status.yaml` producido por sprint planning.

CÓMO FUNCIONA

Tres modos: *interactive*, *data*, *validate*. Seis fases activas: determinar modo, localizar el `sprint-status`, parsear y clasificar, seleccionar próxima acción, mostrar resumen, ofrecer acciones.

CONSULTAR TAMBIÉN

`/bmad-sprint-planning` si el archivo de estado falta o está obsoleto.

2.6 Brownfield — proyectos existentes

Skills específicas para proyectos con código ya implementado. Tres se ejecutan al onboarding el proyecto al método; una se usa para cambios pequeños sin pasar por las cuatro fases.

29

`/bmad-document-project`

● BMad Method

OPC

BROWNFIELD

RESUELVE: Documenta proyectos brownfield para alimentar contexto a agentes IA. Router que delega en sub-workflows: full-scan (rápido / profundo / exhaustivo) o deep-dive (revisión literal sin muestreo).

CUÁNDO INVOCARLA

- Onboarding de un agente IA a un repositorio legacy sin documentación.
- Brownfield, antes de `/bmad-create-architecture`.
- Refresh tras desincronización entre código y docs.

PRODUCE

Documentación en `{project_knowledge}/project-scan-report.json`; `index.md` de navegación.

CUÁNTO TARDA

Quick: 30–60 min. Deep: 2–4 h. Exhaustivo: 4–8 h.

NECESITA ANTES

- Acceso de lectura al repositorio.
- Opcional: estado previo del scan.

CÓMO FUNCIONA

Router en `SKILL.md` + dos sub-workflows (full-scan, deep-dive), cada uno con sus propias instrucciones.

CONSULTAR TAMBIÉN

`/bmad-generate-project-context` después, para destilar las reglas críticas.

30

`/bmad-generate-project-context`

● BMad Method

OPC

BROWNFIELD

RESUELVE: Genera un `project-context.md` conciso con reglas críticas, patrones y guías no obvias que los agentes IA deben recordar al implementar.

CUÁNDO INVOCARLA

- Tras decisiones operativas del equipo (S4 del Libro 3).
- Brownfield, después de `/bmad-document-project`.
- Refresh cuando un patrón se viola repetidamente en code reviews.

PRODUCE

`{output}/project-context.md` optimizado para alimentar el LLM, con frontmatter de seguimiento.

CUÁNTO TARDA

45–75 minutos.

NECESITA ANTES

- Conocimiento del equipo (entrevista guiada).
- Opcional: `project_knowledge/` de `/bmad-document-project`.

CÓMO FUNCIONA

3 sub-pasos: descubrir, generar, completar.

CONSULTAR TAMBIÉN

Skills downstream (`create-story`, `dev-story`, `code-review`) lo leen como contexto.

31

`/bmad-quick-dev`

● BMad Method

OPC

BROWNFIELD

RESUELVE: Workflow unificado intent→código que produce un artefacto reviewable en cinco pasos, sin pasar por PRD, arquitectura ni historias.

CUÁNDO INVOCARLA

- Bug fix puntual, refactor pequeño, retoque de UI o UX.
- Feature menor sin necesidad de PRD.
- Brownfield sin artefactos del método.

PRODUCE

{implementation}/spec-wip.md append-only; código implementado y presentación final (resumen, tests, diff).

CUÁNTO TARDA

20–90 minutos según el alcance.

NECESITA ANTES

- Intent inicial del usuario.
- Opcional: project-context.md y archivos de memoria del agente.

CÓMO FUNCIONA

Cinco pasos numerados (clarify and route, plan, implement, review, present) más un oneshot agresivo. No tiene carpeta steps/ : archivos planos en la raíz.

CONSULTAR TAMBIÉN

No usar para features grandes ni cambios arquitectónicos: ahí entra el flujo completo de la Fase 2.

32 /bmad-qa-generate-e2e-tests ● Test Architect OPC BROWNFIELD

RESUELVE: Generación rápida de tests E2E y de API para código ya implementado (cobertura retroactiva). Auto-detecta el framework existente.

CUÁNDO INVOCARLA

- Brownfield con feature en producción sin tests.
- Tras dev story y code review cuando el dev no produjo todos los tests.

PRODUCE

{implementation}/tests/test-{feature}.spec.ts ya validados; test-summary.md con cobertura y próximos pasos.

CUÁNTO TARDA

20–40 minutos.

NECESITA ANTES

- Código fuente existente.
- package.json con framework detectable (Playwright, Jest, Vitest, Cypress).

CÓMO FUNCIONA

Cinco fases: detectar framework, identificar features, generar tests de API, generar tests E2E, ejecutar y crear resumen. Sin carpeta de sub-pasos.

CONSULTAR TAMBIÉN

No es sustituto de /bmad-code-review ni de /bmad-create-story en modo validate.

3 Apéndice — 34 skills complementarias

Skills que existen en el método pero no forman parte del flujo del proyecto. Aquí aparecen como referencia rápida; no se documentan en detalle porque no se invocan en el ciclo principal.

3.1 Skills BMM auxiliares

Skill	Cuándo	Para qué sirve
/bmad-prfaq	Pre-Fase 1	Forja el concepto de producto con el formato Working Backwards (press release imaginado + FAQ). Alternativa al brief cuando la idea aún no está validada.

↓ continúa en la página siguiente

/bmad-checkpoint-preview	Pre-merge / pre-release	Revisión asistida humano-en-el-bucle para entender un cambio: propósito, contexto, detalles. Útil cuando el revisor llega con poco contexto.
---------------------------------	-------------------------	--

3.2 Agentes con persona — BMM

Seis agentes especializados invocables para conversación abierta en su área de experiencia, sin estructura de workflow.

Skill	Persona	Especialidad
/bmad-agent-analyst	Mary <i>Business Analyst</i>	Análisis estratégico, evidencia verificable, pirámide de Minto.
/bmad-agent-pm	John <i>Product Manager</i>	Jobs-to-be-Done, valor de usuario primero, viabilidad técnica como restricción.
/bmad-agent-architect	Winston <i>System Architect</i>	Trade-offs sobre verdades, productividad del desarrollador como criterio arquitectónico.
/bmad-agent-ux-designer	Sally <i>UX Designer</i>	Empatía con rigor en casos de borde; cada decisión sirve a una necesidad real del usuario.
/bmad-agent-dev	Amelia <i>Senior Software Engineer</i>	Test-first, 100 % pass antes de revisión, comunicación tipo terminal.
/bmad-agent-tech-writer	Paige <i>Technical Writer</i>	Diagramas sobre prosa, cada palabra ganándose su sitio.

3.3 Agente con persona — TEA

Skill	Persona	Especialidad
/bmad-tea	Murat <i>Test Architect</i>	Estrategia basada en riesgo, arquitectura de fixtures, ATDD, contract testing, performance y chaos. Opiniones fuertes, débilmente sostenidas.

3.4 Workflows del módulo CIS

Skill	Cuándo	Para qué sirve
/bmad-cis-design-thinking	Anytime	Procesos centrados en humanos: empatía, definición, ideación, prototipado, test.
/bmad-cis-innovation-strategy	Anytime	Identificar oportunidades de disrupción y diseñar innovación de modelo de negocio.
/bmad-cis-problem-solving	Anytime	Aplicar metodologías sistemáticas de resolución de problemas a desafíos complejos.
/bmad-cis-storytelling	Anytime	Construir narrativas convincentes con frameworks de historia.

3.5 Agentes con persona — CIS

Skill	Persona	Especialidad
<code>/bmad-cis-agent-storyteller</code>	Sophia <i>Master Storyteller</i>	Narrativa con rigor estructural y arco mítico.
<code>/bmad-cis-agent-design-thinking-coach</code>	Maya <i>Design Thinking Maestro</i>	Diseño centrado en humanos; tratar fracaso como feedback.
<code>/bmad-cis-agent-brainstorming-coach</code>	Carson <i>Brainstorming Specialist</i>	Seguridad psicológica para liberar ideas salvajes.
<code>/bmad-cis-agent-creative-problem-solver</code>	Dr. Quinn <i>Master Problem Solver</i>	Sistemas, causa raíz, la pregunta correcta vence a la respuesta rápida.
<code>/bmad-cis-agent-innovation-strategist</code>	Victor <i>Innovation Oracle</i>	Disrupción, estrategia Blue Ocean, modelos de negocio.
<code>/bmad-cis-agent-presentation-master</code>	Caravaggio <i>Presentation Expert</i>	Arquitectura de presentaciones, jerarquía visual, regla de los 3 segundos.

3.6 Skills del módulo BMB

Meta-skills para extender BMAD: crear agentes propios, módulos personalizados o workflows nuevos. Solo aplica a equipos que vayan a personalizar el método.

Skill	Para qué sirve
<code>/bmad-bmb-setup</code>	Configura el módulo BMB en un proyecto.
<code>/bmad-agent-builder</code>	Construye, edita o analiza agentes mediante descubrimiento conversacional.
<code>/bmad-module-builder</code>	Planifica, crea y valida módulos BMAD nuevos.
<code>/bmad-workflow-builder</code>	Construye, convierte y analiza workflows y skills.

3.7 Skills del módulo Core

Utilidades transversales del método. Invocables según necesidad operativa.

Skill	Para qué sirve
<code>/bmad-help</code>	Analiza el estado actual y la consulta del usuario para responder preguntas BMAD o recomendar la siguiente skill.
<code>/bmad-customize</code>	Autora overrides de personalización: hechos persistentes, plantillas, hooks, menús. Escribe en <code>_bmad/custom/</code> y verifica el merge.
<code>/bmad-shard-doc</code>	Divide documentos markdown grandes en archivos más pequeños y organizados por secciones de nivel 2.

↓ continúa en la página siguiente

<code>/bmad-index-docs</code>	Genera o actualiza un <code>index.md</code> que referencia todos los documentos de una carpeta.
<code>/bmad-distillator</code>	Compresión sin pérdida de documentos optimizada para LLM.
<code>/bmad-party-mode</code>	Orquesta discusiones grupales entre agentes BMAD instalados con pensamiento independiente.
<code>/bmad-advanced-elicitation</code>	Empuja al LLM a reconsiderar, refinar y mejorar su output reciente (Socrático, primeros principios, pre-mortem, red team).
<code>/bmad-editorial-review-prose</code>	Revisa un texto frase por frase y detecta problemas de claridad, ambigüedad o redundancia. Devuelve tabla de hallazgos con sugerencias.
<code>/bmad-editorial-review-structure</code>	Editor estructural: propone cortes, reorganización y simplificación preservando comprensión.
<code>/bmad-review-adversarial-general</code>	Revisión cínica de un artefacto y producción de un reporte de hallazgos.
<code>/bmad-review-edge-case-hunter</code>	Recorre toda ruta de bifurcación y condición límite del contenido y reporta solo los casos de borde no manejados.

4 Verificación y fuentes

Fecha de publicación	2026-05-07
Versión BMAD verificada	BMAD v6.6.0 (Core 6.6.0 · BMM 6.6.0 · BMB 1.7.0 · CIS 0.2.0 · TEA 1.15.1)
Total de skills	66. Distribución: BMM 30 (incluye 6 agentes con persona y 2 auxiliares) · Core 12 · TEA 10 (incluye 1 agente Murat) · CIS 10 (incluye 6 agentes) · BMB 4.
Origen de los conteos de sub-pasos	Recuento directo de archivos en <code>.claude/skills/<skill>/steps-c/</code> , <code>steps-e/</code> y <code>steps-v/</code> .
Origen de obligatoriedad y dependencias	Campo <code>required</code> y campo <code>after</code> de <code>_bmad/_config/bmad-help.csv</code> .
Fuentes consultadas	Repositorios oficiales <code>bmad-code-org/bmad-method</code> , <code>bmad-code-org/bmad-method-test-architecture-enterprise</code> , <code>bmad-code-org/bmad-builder</code> , <code>bmad-code-org/bmad-module-creative-intelligence-suite</code> .